```
import os
import re
import time
import json
import pandas as pd
import numpy as np
from collections import Counter
from tqdm import tqdm
from openai import OpenAI
import matplotlib.pyplot as plt
```

```
# API setup
# Put your NVIDIA API key here or set it as an environment variable

os.environ["NVIDIA_API_KEY"] = "YOUR_API_KEY"

client = OpenAI(
    base_url="https://integrate.api.nvidia.com/v1",
    api_key=os.environ["NVIDIA_API_KEY"],
)

MODEL_NAME = "openai/gpt-oss-20b"
```

```
def get_completion(prompt, temperature=0, max_tokens=800):
    try:
        response = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[
                {"role": "user", "content": prompt}
            ],
            temperature=temperature,
            max_tokens=max_tokens
        )

        content = response.choices[0].message.content

        if content is None or content.strip() == "":
            return None

        return content.strip()

    except Exception as e:
        print("Error:", e)
        return None
```

```
def run_reasoning_gene_annotation(up_genes, down_genes, max_retries=3):
    start_time = time.time()

    prompt = f"""
You are a biomedical genomics expert.

Given the following differentially expressed genes from colorectal adenoma:

Upregulated genes:
{", ".join(up_genes)}

Downregulated genes:
{", ".join(down_genes)}

Task:
Generate a structured biomedical interpretation of the gene lists.

STRICT OUTPUT FORMAT:
Return only a markdown table.
The first line must be:
| Pathway_or_Process | Direction | Supporting_Genes | Biological_Rationale | Confidence |

Do not write any text before the table.
Do not write any text after the table.
Do not explain your reasoning process.
Do not include phrases such as "We need to", "Let's propose", or "Potential processes".

Rules:
```

```
- Use only genes provided in the input lists.
- Do not add genes that were not provided.
- Do not claim statistical significance for pathways.
- Avoid broad claims unless supported by multiple genes.
- Confidence must be High, Medium, or Low.
- Return exactly 10 rows only.
"""

    final_raw = None

    for attempt in range(1, max_retries + 1):
        raw = get_completion(prompt, temperature=0, max_tokens=3000)

        if raw is not None and "| Pathway_or_Process |" in raw:
            table_rows = [
                line for line in raw.splitlines()
                if line.strip().startswith("|") and not line.strip().startswith("|---")
            ]

            # header + 10 rows = at least 11 table rows
            if len(table_rows) >= 11:
                final_raw = raw
                break

        print(f"Structured reasoning attempt {attempt} failed. Retrying...")

    if final_raw is None:
        final_raw = "FAILED_TO_GENERATE_STRUCTURED_REASONING_TABLE"

    duration = time.time() - start_time
    return final_raw, duration
```

```
def run_self_consistency_gene_annotation(up_genes, down_genes, k=3, max_retries=12):
    start_time = time.time()
    outputs = []

    input_genes = set(up_genes + down_genes)
    run_number = 1
    total_attempts = 0
    max_total_attempts = k * max_retries

    while len(outputs) < k and total_attempts < max_total_attempts:
        final_raw = None

        prompt = f"""
You are a biomedical genomics expert.

Given these differentially expressed genes from colorectal adenoma:

Upregulated genes in adenoma:
{", ".join(up_genes)}

Downregulated genes in adenoma:
{", ".join(down_genes)}

Task:
Generate 8 to 10 biological themes represented by these genes.

Return ONLY numbered lines.
Do not write introduction.
Do not write explanation before or after the list.

Format:
1. Biological process name: GENE1, GENE2, GENE3
2. Biological process name: GENE1, GENE2, GENE3

Rules:
- Return at least 8 numbered lines.
- Use only genes from the provided gene lists.
- Do not invent genes.
- Each line must contain at least 2 supporting genes.
- Use specific biological names, not Pathway1 or Biological_Process_Name.
- Do not claim statistical significance.
"""

        for attempt in range(1, max_retries + 1):
```

```
            total_attempts += 1

            raw = get_completion(prompt, temperature=0, max_tokens=3000)

            if raw is not None:
                numbered_lines = [
                    line.strip()
                    for line in raw.splitlines()
                    if re.match(r"^\s*\d+[\.\)]", line.strip())
                ]

                valid_lines = []

                for line in numbered_lines:
                    if ":" not in line:
                        continue

                    theme_part, genes_part = line.split(":", 1)

                    genes = [
                        g.strip().replace(".", "").replace(",", "")
                        for g in re.split(r",|;|\s+", genes_part)
                        if g.strip()
                    ]

                    genes_in_input = [
                        g for g in genes
                        if g in input_genes
                    ]

                    if len(genes_in_input) >= 2:
                        clean_line = theme_part.strip() + ": " + ", ".join(genes_in_input)
                        valid_lines.append(clean_line)

                if len(valid_lines) >= 7:
                    final_raw = "\n".join(valid_lines[:10])
                    break

            print(f"Self-consistency valid run {len(outputs)+1}, attempt {attempt} failed. Retrying...")

        if final_raw is not None:
            outputs.append(final_raw)
            print(f"Valid self-consistency run {len(outputs)} generated successfully.")
        else:
            print("One attempted run was skipped because it did not produce valid output.")

    if len(outputs) < k:
        print(f"Warning: only {len(outputs)} valid self-consistency runs were generated.")

    duration = time.time() - start_time
    return outputs, duration
```

```
# Phase 2: Load pre-filtered Top 50 gene lists
# These files should be the SAME gene lists used for GO/KEGG enrichment.

import pandas as pd
import os

up_file = "/content/UP-50-GSE8671.xlsx"
down_file = "/content/DOWN-50-GSE8671.xlsx"

def load_gene_symbols(file_path):
    """
    Load gene symbols from a pre-filtered Top gene list.
    Works with .xlsx, .csv, and .txt/.tsv files.
    """
    ext = os.path.splitext(file_path)[1].lower()

    if ext in [".xlsx", ".xls"]:
        df = pd.read_excel(file_path)
    elif ext == ".csv":
        df = pd.read_csv(file_path)
    else:
        df = pd.read_csv(file_path, sep="\t")

    print(f"\nPreview of {file_path}:")
```

```
    display(df.head())

    # Try to find a gene symbol column
    possible_cols = ["Gene.symbol", "Gene Symbol", "gene.symbol", "gene_symbol", "Symbol", "Gene"]
    gene_col = None

    for col in possible_cols:
        if col in df.columns:
            gene_col = col
            break

    # If no named column exists, use the second column if available, otherwise first column
    if gene_col is None:
        if df.shape[1] > 1:
            gene_series = df.iloc[:, 1]
        else:
            gene_series = df.iloc[:, 0]
    else:
        gene_series = df[gene_col]

    genes = (
        gene_series
        .dropna()
        .astype(str)
        .str.strip()
        .tolist()
    )

    # Remove header-like values and duplicates
    genes = [
        g for g in genes
        if g.lower() not in ["gene.symbol", "gene symbol", "symbol", "gene", "nan", ""]
    ]

    genes = list(dict.fromkeys(genes))
    return genes

top_up_genes = load_gene_symbols(up_file)
top_down_genes = load_gene_symbols(down_file)

print("Number of top upregulated genes:", len(top_up_genes))
print(top_up_genes)

print("\nNumber of top downregulated genes:", len(top_down_genes))
print(top_down_genes)

# Final LLM input
input_genes = set(top_up_genes + top_down_genes)

print("\nTotal unique input genes for LLM:", len(input_genes))
```

```
Preview of /content/UP-50-GSE8671.xlsx:
```

	ID	adj.P.Val	P.Value	t	B	logFC	Gene.symbol	Gene.title
0	222696_at	9.880000e-35	1.810000e-39	29.998043	79.021858	2.589510	AXIN2	axin 2
1	203256_at	2.810000e-31	1.030000e-35	25.930088	70.735590	6.372120	CDH3	cadherin 3
2	212014_x_at	8.610000e-31	5.560000e-35	25.193106	69.105643	2.142557	CD44	CD44 molecule (Indian blood group)
3	212942_s_at	1.030000e-30	9.400000e-35	24.967117	68.597200	6.013322	CEMIP	cell migration inducing hyaluronan binding pro...
4	201563_at	2.070000e-30	2.270000e-34	24.590951	67.741680	1.980909	SORD	sorbitol dehydrogenase

```
Preview of /content/DOWN-50-GSE8671.xlsx:
```

	ID	adj.P.Val	P.Value	t	B	logFC	Gene.symbol	Gene.title
0	206134_at	3.540000e-30	4.540000e-34	-24.299872	67.071657	-3.775792	ADAMDEC1	ADAM like decysin 1
1	204036_at	2.060000e-29	3.390000e-33	-23.469024	65.119619	-2.274632	LPAR1	lysophosphatidic acid receptor 1
2	227354_at	1.210000e-28	2.660000e-32	-22.639991	63.111389	-2.317503	PAG1	phosphoprotein membrane anchor with glycosphin...
3	209735_at	4.050000e-28	9.620000e-32	-22.135556	61.858731	-4.430700	ABCG2	ATP binding cassette subfamily G member 2 (Jun...

```
from pathlib import Path
import re

def extract_dataset_name(file_path, prefix):
    """
    Extract dataset name from file name.

    Examples:
    UP-50-VALIDATION.xlsx   -> VALIDATION
    DOWN-50-VALIDATION.xlsx -> VALIDATION
    UP_50_GSE8671.xlsx      -> GSE8671
    """

    file_name = Path(file_path).stem  # remove extension

    pattern = rf"(?i)^{prefix}[-_]50[-_](.+)$"
    match = re.search(pattern, file_name)

    if not match:
        raise ValueError(
            f"File name '{file_name}' does not follow the required format.\n"
            f"Expected format: {prefix}-50-DATASETNAME.xlsx or {prefix}_50_DATASETNAME.xlsx"
        )

    dataset_name = match.group(1).strip()

    if dataset_name == "":
        raise ValueError(f"No dataset name found in file: {file_name}")

    return dataset_name


# Extract dataset names
up_dataset_name = extract_dataset_name(up_file, "UP")
down_dataset_name = extract_dataset_name(down_file, "DOWN")

# Check if both belong to same dataset
if up_dataset_name != down_dataset_name:
    raise ValueError(
        "UP and DOWN files do not belong to the same dataset!\n"
        f"UP dataset   : {up_dataset_name}\n"
        f"DOWN dataset : {down_dataset_name}"
    )
```

```
# Final dataset name
DATASET_NAME = up_dataset_name
SAFE_DATASET_NAME = DATASET_NAME.lower()

print("Detected dataset name:", DATASET_NAME)
print("Safe dataset name:", SAFE_DATASET_NAME)
print("UP and DOWN files match correctly.")
```

```
Detected dataset name: GSE8671
Safe dataset name: gse8671
UP and DOWN files match correctly.
```

```
def run_gene_annotation_verification(up_genes, down_genes, annotation_output):
    start_time = time.time()

    prompt = f"""
You are a strict biomedical verification assistant.

Input gene lists:

Upregulated genes:
{", ".join(up_genes)}

Downregulated genes:
{", ".join(down_genes)}

Proposed LLM annotation:
{annotation_output}

Task:
Verify each pathway/process in the proposed annotation.

Return ONLY a markdown table.
Do not write any text before or after the table.

The table must have exactly these columns:
Pathway_or_Process | Verification_Status | Problem_Found | Corrected_Comment

Verification_Status must be one of:
Supported, Partially Supported, Unsupported

Rules:
- Check whether the supporting genes are present in the input lists.
- Check whether the gene-pathway relationship is biologically plausible.
- Mark broad or weakly supported claims as Partially Supported.
- Mark unsupported or invented claims as Unsupported.
- Do not add new pathways.
"""

    raw = get_completion(prompt, temperature=0, max_tokens=1800)
    duration = time.time() - start_time

    return raw, duration
```

```
pd.DataFrame({"Gene.symbol": top_up_genes}).to_csv(
    f"input_{SAFE_DATASET_NAME}_top50_upregulated_genes.csv",
    index=False
)

pd.DataFrame({"Gene.symbol": top_down_genes}).to_csv(
    f"input_{SAFE_DATASET_NAME}_top50_downregulated_genes.csv",
    index=False
)

print("Saved input Top 50 gene lists for:", DATASET_NAME)
```

```
Saved input Top 50 gene lists for: GSE8671
```

```
reasoning_output, reasoning_time = run_reasoning_gene_annotation(top_up_genes, top_down_genes)

print("Running time:", round(reasoning_time, 2), "seconds")
print(reasoning_output)
```

```
Running time: 6.89 seconds
| Pathway_or_Process | Direction | Supporting_Genes | Biological_Rationale | Confidence |
|--------------------|-----------|------------------|----------------------|-------------|
```

```
| Wnt signaling activation | Up | AXIN2, RNF43, MYC, CD44 | Upregulation of key Wnt pathway components promotes β-catenin signal
| Cell cycle progression | Up | CDK4, NPM1, MYC, RAN | Elevated cyclin-dependent kinase, nucleophosmin, MYC, and nucleoporin dri
| EMT and cell adhesion | Up | CDH3, CD44, CLDN1, MMP7, S100A11 | Increased adhesion molecules and protease activity indicate ep
| Glycolytic shift | Up | SORD, GART, IMPDH2, MTHFD1L | Upregulation of enzymes involved in glycolysis and nucleotide synthesis
| Stem cell-like phenotype | Up | CD44, FOXQ1, MYC, CDH3 | Co-expression of stemness markers suggests acquisition of a progenito
| Reduced IL-6 signaling | Down | IL6R, PHLPP2, TP53INP2 | Loss of IL-6 receptor and downstream regulators indicates dampened in
| Cholesterol efflux down | Down | ABCA8, ABCG2 | Decreased expression of ATP-binding cassette transporters may impair cholester
| Ion transport down | Down | SLC4A4, SLC17A4, TRPM6, TMEM171 | Downregulation of solute carriers and channels suggests altered
| ECM remodeling | Up | MMP7 | Elevated matrix metalloproteinase promotes extracellular matrix degradation | Low |
| Secretory differentiation down | Down | PYY, HAPLN1, GUCA2A, GUCA2B | Loss of secretory and extracellular matrix proteins refl
```

```
sc_outputs, sc_time = run_self_consistency_gene_annotation(top_up_genes, top_down_genes, k=3)

print("Running time:", round(sc_time, 2), "seconds")

for i, output in enumerate(sc_outputs, 1):
    print(f"\n--- Run {i} ---")
    print(output)
```

```
Valid self-consistency run 1 generated successfully.
Valid self-consistency run 2 generated successfully.
Valid self-consistency run 3 generated successfully.
Running time: 15.7 seconds

--- Run 1 ---
1. Wnt signaling activation: AXIN2, MYC, CD44
2. Cell adhesion and EMT: CDH3, CD44, CLDN1
3. Extracellular matrix remodeling: MMP7, CEMIP
4. Cell proliferation and cell cycle progression: MYC, CDK4, NPM1
5. Metabolic reprogramming: SORD, GART, IMPDH2
6. Ion transport and membrane potential regulation: SLC12A2, TRPM6
7. Immune signaling via cytokine receptor: IL6R, PHLPP2
8. Apoptosis regulation: PHLPP2, TP53INP2
9. Transcription factor-mediated developmental regulation: FOXQ1, TBX3
10. Ribosome biogenesis and protein synthesis: MRPS23, PRMT5

--- Run 2 ---
1. Wnt signaling activation: AXIN2, RNF43, MYC
2. Cell adhesion and EMT: CDH3, CD44, CLDN1, MMP7
3. Cell cycle progression: CDK4, MYC, NPM1
4. Metabolic reprogramming: SORD, GART, IMPDH2
5. Extracellular matrix remodeling: MMP7, CEMIP, S100A11
6. Ion transport regulation: SLC12A2, TRPM6
7. Immune signaling modulation: IL6R, NR3C1
8. Apoptosis regulation: PHLPP2, TP53INP2
9. Glycosylation and protein processing: GALNT6, MGAT4A
10. Transcription factor activity: FOXQ1, TBX3

--- Run 3 ---
1. Wnt signaling pathway activation: AXIN2, MYC
2. Cell adhesion and epithelial-mesenchymal transition: CDH3, CLDN1, CD44, MMP7
3. Extracellular matrix remodeling: MMP7, CEMIP
4. Cell cycle progression: CDK4, MYC, ECT2, NPM1
5. Metabolic reprogramming: SORD, GART, IMPDH2, PPA1
6. Ion transport regulation: SLC12A2, TRPM6
7. Inflammatory signaling modulation: NFE2L3, IL6R
8. Apoptosis regulation: TRIM16, PHLPP2
9. Transcription factor activity: FOXQ1, TBX3, GTF2IRD1
10. Ribosome biogenesis: MRPS23, PRMT5, NUP37
```

```
results = {
    "dataset": DATASET_NAME,
    "input_summary": {
        "top_upregulated_genes": len(top_up_genes),
        "top_downregulated_genes": len(top_down_genes),
        "total_input_genes": len(top_up_genes) + len(top_down_genes),
        "total_unique_input_genes": len(set(top_up_genes + top_down_genes))
    },
    "top_up_genes": top_up_genes,
    "top_down_genes": top_down_genes,
    "reasoning_output": reasoning_output,
    "reasoning_time": reasoning_time,
    "self_consistency_outputs": sc_outputs,
    "self_consistency_time": sc_time,
}

json_filename = f"llm_gene_reasoning_outputs_{SAFE_DATASET_NAME}_top50.json"

with open(json_filename, "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=4)
```

```
print("Saved JSON file:", json_filename)
```

```
Saved JSON file: llm_gene_reasoning_outputs_gse8671_top50.json
```

```
with open(json_filename, "r", encoding="utf-8") as f:
    saved_results = json.load(f)

print("Dataset:", saved_results["dataset"])
print(saved_results["reasoning_output"])
```

```
Dataset: GSE8671
| Pathway_or_Process | Direction | Supporting_Genes | Biological_Rationale | Confidence |
|---------------------|-----------|------------------|----------------------|-------------|
| Wnt signaling activation | Up | AXIN2, RNF43, MYC, CD44 | Upregulation of key Wnt pathway components promotes β-catenin signal
| Cell cycle progression | Up | CDK4, NPM1, MYC, RAN | Elevated cyclin-dependent kinase, nucleophosmin, MYC, and nucleoporin dri
| EMT and cell adhesion | Up | CDH3, CD44, CLDN1, MMP7, S100A11 | Increased adhesion molecules and protease activity indicate ep
| Glycolytic shift | Up | SORD, GART, IMPDH2, MTHFD1L | Upregulation of enzymes involved in glycolysis and nucleotide synthesis
| Stem cell-like phenotype | Up | CD44, FOXQ1, MYC, CDH3 | Co-expression of stemness markers suggests acquisition of a progenito
| Reduced IL-6 signaling | Down | IL6R, PHLPP2, TP53INP2 | Loss of IL-6 receptor and downstream regulators indicates dampened in
| Cholesterol efflux down | Down | ABCA8, ABCG2 | Decreased expression of ATP-binding cassette transporters may impair cholester
| Ion transport down | Down | SLC4A4, SLC17A4, TRPM6, TMEM171 | Downregulation of solute carriers and channels suggests altered
| ECM remodeling | Up | MMP7 | Elevated matrix metalloproteinase promotes extracellular matrix degradation | Low |
| Secretory differentiation down | Down | PYY, HAPLN1, GUCA2A, GUCA2B | Loss of secretory and extracellular matrix proteins refl
```

```
def run_direct_gene_annotation(up_genes, down_genes):
    start_time = time.time()

    prompt = f"""
You are a biomedical genomics expert.

You will receive two gene lists from differential expression analysis comparing
normal colorectal mucosa and colorectal adenoma.

Upregulated genes in adenoma:
{", ".join(up_genes)}

Downregulated genes in adenoma:
{", ".join(down_genes)}

Task:
Identify the main biological pathways or processes represented by these genes.

Return ONLY a markdown table.
Do not write explanations before or after the table.
Do not write your reasoning process.

The table must have exactly these columns:
Pathway_or_Process | Direction | Supporting_Genes | Explanation | Confidence

Rules:
- Use only genes from the provided lists.
- Do not invent genes.
- Do not claim causality.
- Do not claim statistical significance for pathways.
- Keep each explanation one sentence only.
- Confidence must be High, Medium, or Low.
- Return exactly 10 rows only.
"""

    raw = get_completion(prompt, temperature=0, max_tokens=3000)
    duration = time.time() - start_time

    return raw, duration
```

Phase 6: Additional Evaluation Metrics

Phase 6.2: Hallucinated Gene Check

```
direct_output, direct_time = run_direct_gene_annotation(top_up_genes, top_down_genes)
```

```
print("Direct annotation running time:", round(direct_time, 2), "seconds")
print(direct_output)
```

```
Direct annotation running time: 6.5 seconds
| Pathway_or_Process | Direction | Supporting_Genes | Explanation | Confidence |
|---|---|---|---|---|
| Wnt signaling activation | Upregulated | AXIN2, MYC, RNF43 | Upregulation of Wnt pathway components indicates increased signal
| Cell cycle progression | Upregulated | CDK4, MYC, NPM1, RAN, NUP37 | Genes involved in cell cycle regulation are up, suggestin
| EMT / cell adhesion changes | Upregulated | CDH3, CLDN1, CD44, CEMIP, MMP7 | Upregulation of adhesion and ECM remodeling genes
| Metabolic reprogramming | Upregulated | GART, IMPDH2, SORD, MTHFD1L, TRAP1 | Upregulation of metabolic enzymes indicates alter
| Secretory function downregulation | Downregulated | PYY, GUCA2A, GUCA2B, AQP8, SLC4A4 | Loss of secretory and transport genes
| Immune signaling downregulation | Downregulated | IL6R, ADAMDEC1, LPAR1, PAG1 | Reduced expression of cytokine receptors sugge
| Ion transport downregulation | Downregulated | SLC4A4, SLC17A4, TRPM6, TMCC3 | Downregulation of ion transporters indicates im
| Apoptosis regulation downregulation | Downregulated | PHLPP2, TP53INP2, MEIS1 | Decreased expression of pro-apoptotic regulato
| ECM remodeling | Upregulated | MMP7, CEMIP, S100A11, TEAD4 | Upregulation of ECM remodeling genes indicates matrix changes. |
| Stemness / progenitor expansion | Upregulated | CD44, AXIN2, MYC | Upregulation of stemness markers suggests progenitor cell e
```

```
# Phase 6.3: Method Comparison Table with Runtime

method_comparison_df = pd.DataFrame([
    {
        "Method": "Direct Gene Annotation",
        "Output_Type": "Biological themes table",
        "Reasoning_Component": "Low",
        "Verification_Used": "No",
        "Runtime_seconds": round(direct_time, 2) if "direct_time" in globals() else "Not recorded",
        "Main_Strength": "Fast baseline interpretation"
    },
    {
        "Method": "Structured Reasoning Annotation",
        "Output_Type": "Themes with supporting genes, rationale, and confidence",
        "Reasoning_Component": "High",
        "Verification_Used": "No",
        "Runtime_seconds": round(reasoning_time, 2) if "reasoning_time" in globals() else "Not recorded",
        "Main_Strength": "Provides interpretable biological rationale"
    },
    {
        "Method": "Self-Consistency",
        "Output_Type": "Repeated candidate biological themes",
        "Reasoning_Component": "Medium",
        "Verification_Used": "Partial",
        "Runtime_seconds": round(sc_time, 2) if "sc_time" in globals() else "Not recorded",
        "Main_Strength": "Tests stability of repeated LLM outputs"
    },
    {
        "Method": "Verification-Enhanced Reasoning",
        "Output_Type": "Verification status for each LLM theme",
        "Reasoning_Component": "High",
        "Verification_Used": "Yes",
        "Runtime_seconds": round(verification_time, 2) if "verification_time" in globals() else "Not recorded",
        "Main_Strength": "Checks gene presence and biological plausibility"
    }
])

method_comparison_df
```

	Method	Output_Type	Reasoning_Component	Verification_Used	Runtime_seconds	Main_Strength
0	Direct Gene Annotation	Biological themes table	Low	No	6.5	Fast baseline interpretation
1	Structured Reasoning Annotation	Themes with supporting genes, rationale, and c...	High	No	6.89	Provides interpretable biological rationale
		Repeated candidate				Tests stability of repeated

Next steps: Generate code with `method_comparison_df` New interactive sheet

Phase 6.4: Best-of-K Reasoning Selection

```
# Phase 6.4: API-Based Best-of-K Reasoning Selection

import json
import re
import time
```

```
import pandas as pd

def parse_json_output(raw_output):
    """
    Extract JSON list from LLM output.
    If JSON parsing fails, try to parse Theme/Supporting genes text format.
    """
    if raw_output is None:
        return None

    cleaned = raw_output.strip()

    # Remove markdown fences if present
    cleaned = cleaned.replace("```json", "").replace("```", "").strip()

    # Try to extract valid JSON array
    start = cleaned.find("[")
    end = cleaned.rfind("]")

    if start != -1 and end != -1 and end > start:
        json_part = cleaned[start:end+1]
        try:
            parsed = json.loads(json_part)
            if isinstance(parsed, list):
                return parsed
        except Exception:
            pass

    # Fallback parser if model returns text like:
    # Theme 1: "..."
    # Supporting genes: GENE1, GENE2
    fallback_items = []

    theme_pattern = r'Theme\s*\d+\s*:\s*"?([^"\n]+)"?'
    themes = re.findall(theme_pattern, cleaned, flags=re.IGNORECASE)

    gene_pattern = r'Supporting genes\s*:\s*([A-Za-z0-9_,\-\s]+)'
    gene_groups = re.findall(gene_pattern, cleaned, flags=re.IGNORECASE)

    if themes:
        for i, theme in enumerate(themes[:10]):
            genes = []
            if i < len(gene_groups):
                genes = [
                    g.strip()
                    for g in re.split(r",|\s+", gene_groups[i])
                    if g.strip() and len(g.strip()) > 1
                ]

            fallback_items.append({
                "theme": theme.strip(),
                "direction": "Mixed",
                "supporting_genes": genes[:5]
            })

        if fallback_items:
            return fallback_items

    print("Parsing failed. Raw output:")
    print(raw_output)
    return None


def generate_candidate_gene_themes(up_genes, down_genes, candidate_id):
    """
    Generate one candidate set of biological themes using the LLM.
    """
    prompt = f"""
You are a biomedical genomics expert.

Given these differentially expressed genes from colorectal adenoma:

Upregulated genes in adenoma:
{", ".join(up_genes)}

Downregulated genes in adenoma:
{", ".join(down_genes)}
```

```
Task:
Generate exactly 10 biological themes represented by these genes.

Return valid JSON only.
Do not write any text before or after the JSON.
Do not use markdown fences.

JSON format:
[
  {{
    "theme": "theme name",
    "direction": "Upregulated",
    "supporting_genes": ["GENE1", "GENE2", "GENE3"]
  }}
]

Rules:
- Return exactly 10 objects.
- Use only genes from the provided gene lists.
- Do not invent genes.
- Each theme must use 2 to 5 supporting genes only.
- Theme names must be specific and biologically informative.
- Direction must be one of: Upregulated, Downregulated, Mixed.
"""
    start_time = time.time()
    raw = get_completion(prompt, temperature=0, max_tokens=3000)
    runtime = time.time() - start_time

    parsed = parse_json_output(raw)

    return {
        "candidate_id": candidate_id,
        "raw_output": raw,
        "parsed_output": parsed,
        "runtime_seconds": runtime
    }


def classify_theme_against_go_kegg(theme):
    """
    Simple rule-based mapping based on the final GO/KEGG comparison matrix.
    """
    theme_lower = theme.lower()

    if "wnt" in theme_lower or "β-catenin" in theme_lower or "beta-catenin" in theme_lower:
        return "Partial"

    if (
        "adhesion" in theme_lower or
        "emt" in theme_lower or
        "junction" in theme_lower or
        "ecm" in theme_lower or
        "remodeling" in theme_lower or
        "invasion" in theme_lower
    ):
        return "Match"

    if (
        "cell cycle" in theme_lower or
        "cell-cycle" in theme_lower or
        "proliferation" in theme_lower or
        "mitotic" in theme_lower
    ):
        return "Match"

    if (
        "metabolic" in theme_lower or
        "metabolism" in theme_lower or
        "glycolysis" in theme_lower or
        "fructose" in theme_lower or
        "amino acid" in theme_lower
    ):
        return "Partial"

    if (
        "barrier" in theme_lower or
        "transport" in theme_lower or
```

```
        "epithelial" in theme_lower or
        "ion" in theme_lower
    ):
        return "Partial"

    return "Unsupported"


def calculate_candidate_metrics(candidate, input_genes):
    """
    Calculate descriptive candidate metrics without converting
    Match/Partial/Unsupported into 2/1/0 scores.
    """
    parsed = candidate["parsed_output"]

    if parsed is None:
        return {
            "Candidate": candidate["candidate_id"],
            "Number_of_Themes": 0,
            "Match_Count": 0,
            "Partial_Count": 0,
            "Unsupported_Count": 0,
            "Match_Rate_%": 0,
            "Partial_or_Match_Rate_%": 0,
            "Unsupported_Rate_%": 100,
            "Hallucination_Rate_%": 100,
            "Runtime_seconds": round(candidate["runtime_seconds"], 2),
            "Themes": "Parsing failed"
        }

    statuses = []
    all_used_genes = []
    themes = []

    for item in parsed:
        theme = item.get("theme", "")
        genes = item.get("supporting_genes", [])

        if isinstance(genes, str):
            genes = [g.strip() for g in genes.split(",")]

        themes.append(theme)
        all_used_genes.extend(genes)

        status = classify_theme_against_go_kegg(theme)
        statuses.append(status)

    total_themes = len(themes)

    match_count = statuses.count("Match")
    partial_count = statuses.count("Partial")
    unsupported_count = statuses.count("Unsupported")

    hallucinated_genes = [
        gene for gene in all_used_genes
        if gene not in input_genes
    ]

    hallucination_rate = (
        len(hallucinated_genes) / len(all_used_genes) * 100
    ) if all_used_genes else 100

    return {
        "Candidate": candidate["candidate_id"],
        "Number_of_Themes": total_themes,
        "Match_Count": match_count,
        "Partial_Count": partial_count,
        "Unsupported_Count": unsupported_count,
        "Match_Rate_%": round(match_count / total_themes * 100, 2) if total_themes else 0,
        "Partial_or_Match_Rate_%": round((match_count + partial_count) / total_themes * 100, 2) if total_themes else 0,
        "Unsupported_Rate_%": round(unsupported_count / total_themes * 100, 2) if total_themes else 100,
        "Hallucination_Rate_%": round(hallucination_rate, 2),
        "Runtime_seconds": round(candidate["runtime_seconds"], 2),
        "Themes": "; ".join(themes)
    }
```

```
# Run API-based Best-of-K safely
# Only valid parsed candidates are allowed to enter the final selection

K = 3
max_retries = 5
input_genes = set(top_up_genes + top_down_genes)

api_candidates = []
failed_candidates = []

for i in range(1, K + 1):
    candidate_id = f"Candidate_{i}"
    print(f"Generating {candidate_id}...")

    valid_candidate = None

    for attempt in range(1, max_retries + 1):
        candidate = generate_candidate_gene_themes(
            top_up_genes,
            top_down_genes,
            candidate_id=candidate_id
        )

        parsed = candidate.get("parsed_output")

        if parsed is not None and isinstance(parsed, list) and len(parsed) == 10:
            valid_candidate = candidate
            print(f"{candidate_id} parsed successfully on attempt {attempt}.")
            break

        print(f"{candidate_id} parsing failed on attempt {attempt}. Retrying...")

    if valid_candidate is not None:
        api_candidates.append(valid_candidate)
    else:
        failed_candidates.append(candidate_id)
        print(f"{candidate_id} was skipped because it did not produce valid parsed output.")

if len(api_candidates) == 0:
    raise RuntimeError(
        "No valid Best-of-K candidates were generated. Re-run this cell or increase max_retries."
    )

# Descriptive candidate comparison without 2/1/0 scoring
api_best_of_k_results = []

for candidate in api_candidates:
    result = calculate_candidate_metrics(candidate, input_genes)
    api_best_of_k_results.append(result)

api_best_of_k_df = pd.DataFrame(api_best_of_k_results)

best_api_candidate = api_best_of_k_df.sort_values(
    by=[
        "Hallucination_Rate_%",
        "Unsupported_Count",
        "Match_Count",
        "Partial_or_Match_Rate_%"
    ],
    ascending=[True, True, False, False]
).iloc[0]

selected_candidate_id = best_api_candidate["Candidate"]

selected_candidate = None
for candidate in api_candidates:
    if candidate["candidate_id"] == selected_candidate_id:
        selected_candidate = candidate
        break

if selected_candidate is None:
    raise ValueError("Selected candidate was not found among valid api_candidates.")

selected_parsed_output = selected_candidate["parsed_output"]

if selected_parsed_output is None:
    raise ValueError("Selected candidate still has no parsed output.")
```

```
print("API Best-of-K selected output:", selected_candidate_id)
print("Selection rule: lowest hallucination, lowest unsupported count, highest match count.")

if failed_candidates:
    print("Skipped failed candidates:", failed_candidates)

display(api_best_of_k_df)
```

```
Generating Candidate_1...
Candidate_1 parsed successfully on attempt 1.
Generating Candidate_2...
Candidate_2 parsed successfully on attempt 1.
Generating Candidate_3...
Candidate_3 parsed successfully on attempt 1.
API Best-of-K selected output: Candidate_2
Selection rule: lowest hallucination, lowest unsupported count, highest match count.
```

	Candidate	Number_of_Themes	Match_Count	Partial_Count	Unsupported_Count	Match_Rate_%	Partial_or_Match_Rate_%	Unsupport
0	Candidate_1	10	2	6	2	20.0	80.0	
1	Candidate_2	10	4	5	1	40.0	90.0	
2	Candidate_3	10	3	6	1	30.0	90.0	

Next steps: Generate code with `api_best_of_k_df` New interactive sheet

```
# Phase 6.4b: Automatic Hallucination Check from selected API Best-of-K candidate

import re
import pandas as pd

# Get selected candidate ID from best-of-k result
selected_candidate_id = best_api_candidate["Candidate"]

# Find the selected candidate object inside api_candidates
selected_candidate = None

for candidate in api_candidates:
    if candidate["candidate_id"] == selected_candidate_id:
        selected_candidate = candidate
        break

if selected_candidate is None:
    raise ValueError("Selected candidate was not found in api_candidates.")

selected_parsed_output = selected_candidate["parsed_output"]

if selected_parsed_output is None:
    raise ValueError("Selected candidate has no parsed output.")

# Build llm_theme_genes automatically from selected candidate
llm_theme_genes = {}

for item in selected_parsed_output:
    theme = str(item.get("theme", "")).strip()
    genes = item.get("supporting_genes", [])

    # If genes came as one string, split them
    if isinstance(genes, str):
        genes = re.split(r",|;|\s+", genes)

    # Clean gene names
    cleaned_genes = []
    for gene in genes:
        gene = str(gene).strip()
        gene = gene.replace("[", "").replace("]", "")
        gene = gene.replace('"', "").replace("'", "")
        gene = gene.strip()

        if gene:
            cleaned_genes.append(gene)
```

```
    llm_theme_genes[theme] = cleaned_genes

print("Selected Candidate:", selected_candidate_id)

print("\nAutomatically extracted LLM theme genes:")
display(pd.DataFrame([
    {
        "LLM_Theme": theme,
        "Supporting_Genes": ", ".join(genes)
    }
    for theme, genes in llm_theme_genes.items()
]))

# Hallucination check
input_genes = set(top_up_genes + top_down_genes)

hallucination_results = []

for theme, genes in llm_theme_genes.items():
    hallucinated_genes = [gene for gene in genes if gene not in input_genes]
    hallucination_rate = len(hallucinated_genes) / len(genes) * 100 if genes else 0

    hallucination_results.append({
        "LLM_Theme": theme,
        "Total_Genes_Used": len(genes),
        "Hallucinated_Genes": hallucinated_genes if hallucinated_genes else "None",
        "Hallucination_Rate_%": round(hallucination_rate, 2)
    })

hallucination_df = pd.DataFrame(hallucination_results)

overall_genes_used = sum(len(genes) for genes in llm_theme_genes.values())

overall_hallucinated = sum(
    len([gene for gene in genes if gene not in input_genes])
    for genes in llm_theme_genes.values()
)

overall_hallucination_rate = (
    overall_hallucinated / overall_genes_used * 100
) if overall_genes_used else 0

print("\nOverall Hallucination Rate:", round(overall_hallucination_rate, 2), "%")

display(hallucination_df)
```

```
Selected Candidate: Candidate_2

Automatically extracted LLM theme genes:
```

	LLM_Theme	Supporting_Genes
0	Wnt signaling activation	AXIN2, RNF43, MYC, TBX3
1	Epithelial-mesenchymal transition and invasion	CDH3, CD44, MMP7, S100A11
2	Cell cycle progression and proliferation	CDK4, NPM1, MYC, ECT2
3	Stem-cell-like properties and self-renewal	CD44, AXIN2, MYC, FOXQ1
4	Inflammatory cytokine signaling downregulation	IL6R, PHLPP2, ADAMDEC1, LPAR1
5	Metabolic reprogramming and nucleotide synthesis	GART, IMPDH2, MTHFD1L, TRAP1
6	Cell adhesion and tight junction integrity	CLDN1, CDH3, CD44
7	Extracellular matrix remodeling and invasion	MMP7, CEMIP, SORD, S100A11
8	Ion transport dysregulation	SLC12A2, SLC4A4
9	Transcription factor dysregulation in adenoma	FOXQ1, TBX3, GTF2IRD1, MYC

```
Overall Hallucination Rate: 0.0 %
```

	LLM_Theme	Total_Genes_Used	Hallucinated_Genes	Hallucination_Rate_%
0	Wnt signaling activation	4	None	0.0
1	Epithelial-mesenchymal transition and invasion	4	None	0.0
2	Cell cycle progression and proliferation	4	None	0.0
3	Stem-cell-like properties and self-renewal	4	None	0.0
4	Inflammatory cytokine signaling downregulation	4	None	0.0
5	Metabolic reprogramming and nucleotide synthesis	4	None	0.0
6	Cell adhesion and tight junction integrity	3	None	0.0
7	Extracellular matrix remodeling and invasion	4	None	0.0
8	Ion transport dysregulation	2	None	0.0
9	Transcription factor dysregulation in adenoma	4	None	0.0

Next steps: Generate code with `hallucination_df` New interactive sheet

```
# Phase 6.4c: REAL GO/KEGG Agreement Scoring using enrichment files

import pandas as pd
import numpy as np
import re
import os

# Automatically build GO/KEGG filenames from DATASET_NAME
GO_UP_FILE = f"/content/GO-up-{DATASET_NAME}.txt"
GO_DOWN_FILE = f"/content/GO-down-{DATASET_NAME}.txt"
KEGG_UP_FILE = f"/content/KEGG-up-{DATASET_NAME}.txt"
KEGG_DOWN_FILE = f"/content/KEGG-down-{DATASET_NAME}.txt"



def find_column(df, possible_names):
    for name in possible_names:
        if name in df.columns:
            return name
    return None

def parse_genes(value):
    if pd.isna(value):
        return []
    value = str(value)
    genes = re.split(r"[;,/\s]+", value)
    genes = [g.strip() for g in genes if g.strip()]
    return genes

def load_enrichment_file(file_path, source_name, direction):
```

```
        df = pd.read_csv(file_path, sep="\t")

        term_col = find_column(df, ["Term", "Description", "Pathway", "Name"])
        genes_col = find_column(df, ["Genes", "genes", "Gene.symbol", "geneID", "gene"])
        adj_col = find_column(df, ["Adjusted P-value", "Adjusted.P.Value", "adj.P.Val", "FDR", "qvalue"])

        if term_col is None:
            raise ValueError(f"No term column found in {file_path}")
        if genes_col is None:
            raise ValueError(f"No genes column found in {file_path}")
        if adj_col is None:
            raise ValueError(f"No adjusted p-value column found in {file_path}")

        out = df.copy()
        out["Source"] = source_name
        out["Enrichment_Direction"] = direction
        out["Term_Text"] = out[term_col].astype(str)
        out["Enrichment_Genes_List"] = out[genes_col].apply(parse_genes)
        out["Adjusted_P_value"] = pd.to_numeric(out[adj_col], errors="coerce")

        return out

    # Load real GO/KEGG files
    go_up = load_enrichment_file(GO_UP_FILE, "GO Upregulated", "Upregulated")
    go_down = load_enrichment_file(GO_DOWN_FILE, "GO Downregulated", "Downregulated")
    kegg_up = load_enrichment_file(KEGG_UP_FILE, "KEGG Upregulated", "Upregulated")
    kegg_down = load_enrichment_file(KEGG_DOWN_FILE, "KEGG Downregulated", "Downregulated")

    all_enrichment_real = pd.concat(
        [go_up, go_down, kegg_up, kegg_down],
        ignore_index=True
    )

    def tokenize_text(text):
        text = str(text).lower()
        tokens = re.findall(r"[a-zA-Z0-9]+", text)

        stopwords = {
            "and", "or", "of", "in", "to", "the", "a", "an",
            "process", "pathway", "signaling", "regulation",
            "positive", "negative", "response"
        }

        return set([t for t in tokens if t not in stopwords and len(t) > 2])

    def expand_theme_keywords(theme):
        tokens = tokenize_text(theme)
        expanded = set(tokens)

        theme_lower = str(theme).lower()

        if "cell cycle" in theme_lower or "mitotic" in theme_lower or "proliferation" in theme_lower:
            expanded.update(["cell", "cycle", "mitotic", "mitosis", "division", "proliferation"])

        if "dna" in theme_lower or "replication" in theme_lower:
            expanded.update(["dna", "replication", "nucleotide", "synthesis"])

        if "apoptosis" in theme_lower or "survival" in theme_lower:
            expanded.update(["apoptosis", "apoptotic", "survival", "death"])

        if "immune" in theme_lower or "inflammatory" in theme_lower or "inflammation" in theme_lower:
            expanded.update(["immune", "inflammatory", "inflammation", "cytokine", "leukocyte", "chemokine"])

        if "extracellular matrix" in theme_lower or "ecm" in theme_lower or "adhesion" in theme_lower or "invasion" in theme_lower:
            expanded.update(["extracellular", "matrix", "adhesion", "migration", "invasion", "collagen"])

        if "metabolic" in theme_lower or "metabolism" in theme_lower or "oxidative" in theme_lower:
            expanded.update(["metabolic", "metabolism", "oxidative", "redox", "biosynthesis"])

        if "angiogenesis" in theme_lower or "vascular" in theme_lower:
            expanded.update(["angiogenesis", "vascular", "vessel", "blood"])

        if "wnt" in theme_lower:
            expanded.update(["wnt", "catenin"])

        return expanded
```

```
def compare_theme_to_real_go_kegg(theme, supporting_genes, enrichment_df, adj_p_cutoff=0.10):
    theme_keywords = expand_theme_keywords(theme)

    if isinstance(supporting_genes, str):
        llm_genes = parse_genes(supporting_genes)
    else:
        llm_genes = [str(g).strip() for g in supporting_genes if str(g).strip()]

    llm_genes_set = set(llm_genes)

    best_record = None
    best_score = -1

    for _, row in enrichment_df.iterrows():
        term = row["Term_Text"]
        term_tokens = tokenize_text(term)
        enrichment_genes = set(row["Enrichment_Genes_List"])
        adj_p = row["Adjusted_P_value"]

        term_overlap = theme_keywords.intersection(term_tokens)
        gene_overlap = llm_genes_set.intersection(enrichment_genes)

        term_score = len(term_overlap)
        gene_score = len(gene_overlap) * 2

        # Similarity is based only on gene overlap and term-keyword overlap.
# Adjusted p-value is kept for reporting, not for similarity scoring.
        total_score = term_score + gene_score

        if total_score > best_score:
            best_score = total_score
            best_record = {
                "Best_Enrichment_Source": row["Source"],
                "Best_Enrichment_Term": term,
                "Best_Adjusted_P_value": adj_p,
                "Overlapping_Genes": sorted(list(gene_overlap)),
                "Term_Keyword_Overlap": sorted(list(term_overlap)),
                "Term_Overlap_Count": term_score,
                "Gene_Overlap_Count": len(gene_overlap),
                "Total_Match_Score": total_score
            }

    if best_record is None:
        return {
            "GO_KEGG_Match_Status": "Unsupported",
            "Interpretation": "No GO/KEGG enrichment record was available for comparison."
        }

    gene_overlap_count = best_record["Gene_Overlap_Count"]
    term_overlap_count = best_record["Term_Overlap_Count"]
    adj_p = best_record["Best_Adjusted_P_value"]

    if (
        gene_overlap_count >= 2
        and term_overlap_count >= 1
        and not pd.isna(adj_p)
        and adj_p <= adj_p_cutoff
    ):
        status = "Match"
        interpretation = "The theme matched a real GO/KEGG enrichment term using both keyword overlap and supporting-gene overl
    elif gene_overlap_count >= 1 or term_overlap_count >= 1:
        status = "Partial"
        interpretation = "The theme showed partial support from real GO/KEGG enrichment terms, but the overlap was limited or i
    else:
        status = "Unsupported"
        interpretation = "No clear support was found in the real GO/KEGG enrichment results."

    best_record["GO_KEGG_Match_Status"] = status
    best_record["Interpretation"] = interpretation
    return best_record

# Build real GO/KEGG categorical comparison table for selected Best-of-K candidate
comparison_records = []

for item in selected_parsed_output:
    theme = item.get("theme", "")
    genes = item.get("supporting_genes", [])
```

```
    direction = item.get("direction", "Not specified")

    match_result = compare_theme_to_real_go_kegg(
        theme=theme,
        supporting_genes=genes,
        enrichment_df=all_enrichment_real,
        adj_p_cutoff=0.10
    )

    status = match_result["GO_KEGG_Match_Status"]

    comparison_records.append({
        "LLM_Theme": theme,
        "Direction": direction,
        "Supporting_Genes": ", ".join(genes) if isinstance(genes, list) else genes,
        "GO_KEGG_Match_Status": status,
        "Best_Enrichment_Source": match_result.get("Best_Enrichment_Source", "None"),
        "Best_Enrichment_Term": match_result.get("Best_Enrichment_Term", "None"),
        "Best_Adjusted_P_value": match_result.get("Best_Adjusted_P_value", np.nan),
        "Overlapping_Genes": ", ".join(match_result.get("Overlapping_Genes", [])),
        "Gene_Overlap_Count": match_result.get("Gene_Overlap_Count", 0),
        "Term_Keyword_Overlap": ", ".join(match_result.get("Term_Keyword_Overlap", [])),
        "Term_Overlap_Count": match_result.get("Term_Overlap_Count", 0),
        "Interpretation": match_result["Interpretation"]
    })

comparison_df = pd.DataFrame(comparison_records)

go_kegg_summary_df = pd.DataFrame([{
    "Selected_Candidate": selected_candidate_id,
    "Total_Themes": len(comparison_df),
    "Match_Count": (comparison_df["GO_KEGG_Match_Status"] == "Match").sum(),
    "Partial_Count": (comparison_df["GO_KEGG_Match_Status"] == "Partial").sum(),
    "Unsupported_Count": (comparison_df["GO_KEGG_Match_Status"] == "Unsupported").sum()
}])

go_kegg_summary_df["Partial_or_Match_Count"] = (
    go_kegg_summary_df["Match_Count"] +
    go_kegg_summary_df["Partial_Count"]
)

go_kegg_summary_df["Match_Rate_%"] = (
    go_kegg_summary_df["Match_Count"] /
    go_kegg_summary_df["Total_Themes"] * 100
).round(2)

go_kegg_summary_df["Partial_or_Match_Rate_%"] = (
    go_kegg_summary_df["Partial_or_Match_Count"] /
    go_kegg_summary_df["Total_Themes"] * 100
).round(2)

go_kegg_summary_df["Unsupported_Rate_%"] = (
    go_kegg_summary_df["Unsupported_Count"] /
    go_kegg_summary_df["Total_Themes"] * 100
).round(2)

print("Selected Candidate:", selected_candidate_id)
print("GO/KEGG categorical support summary:")
display(go_kegg_summary_df)

print("Detailed GO/KEGG comparison:")
display(comparison_df)

comparison_df.to_csv(
    f"real_go_kegg_categorical_comparison_{SAFE_DATASET_NAME}.csv",
    index=False
)

go_kegg_summary_df.to_csv(
    f"real_go_kegg_categorical_summary_{SAFE_DATASET_NAME}.csv",
    index=False
)

print("Saved categorical GO/KEGG comparison files.")
```

```
Selected Candidate: Candidate_2
GO/KEGG categorical support summary:
```

	Selected_Candidate	Total_Themes	Match_Count	Partial_Count	Unsupported_Count	Partial_or_Match_Count	Match_Rate_%	Partia
0	Candidate_2	10	1	9	0	10	10.0	

```
Detailed GO/KEGG comparison:
```

	LLM_Theme	Direction	Supporting_Genes	GO_KEGG_Match_Status	Best_Enrichment_Source	Best_Enrichment_Term	Best
0	Wnt signaling activation	Upregulated	AXIN2, RNF43, MYC, TBX3	Partial	GO Upregulated	Female Genitalia Development (GO:0030540)	
1	Epithelial-mesenchymal transition and invasion	Upregulated	CDH3, CD44, MMP7, S100A11	Partial	GO Upregulated	Positive Regulation of Smooth Muscle Cell Migr...	
2	Cell cycle progression and proliferation	Upregulated	CDK4, NPM1, MYC, ECT2	Partial	GO Upregulated	Positive Regulation of Cell Division (GO:0051781)	
3	Stem-cell-like properties and self-renewal	Upregulated	CD44, AXIN2, MYC, FOXQ1	Partial	GO Upregulated	Positive Regulation of Protein Localization to...	
4	Inflammatory cytokine signaling downregulation	Downregulated	IL6R, PHLPP2, ADAMDEC1, LPAR1	Partial	KEGG Downregulated	PI3K-AKT SIGNALING PATHWAY	
5	Metabolic reprogramming and nucleotide synthesis	Upregulated	GART, IMPDH2, MTHFD1L, TRAP1	Match	KEGG Upregulated	PURINE METABOLISM	
6	Cell adhesion and tight junction integrity	Upregulated	CLDN1, CDH3, CD44	Partial	GO Upregulated	Bicellular Tight Junction Assembly (GO:0070830)	
7	Extracellular matrix remodeling and invasion	Upregulated	MMP7, CEMIP, SORD, S100A11	Partial	GO Upregulated	Positive Regulation of Smooth Muscle Cell Migr...	
8	Ion transport dysregulation	Mixed	SLC12A2, SLC4A4	Partial	GO Downregulated	Metal Ion Transport (GO:0030001)	
9	Transcription factor dysregulation in adenoma	Upregulated	FOXQ1, TBX3, GTF2IRD1, MYC	Partial	GO Upregulated	Regulation of DNA-templated Transcription (GO:...	

```
Saved categorical GO/KEGG comparison files.
```

Next steps: Generate code with `comparison_df` | New interactive sheet

```
# Phase 6.4d: LLM-predicted GO/KEGG-like profiles
# Goal:
# Ask the LLM to predict Top 10 GO-like and KEGG-like results
# separately for upregulated and downregulated genes,
# then compare them with real GO/KEGG enrichment results.

import re
import time
import pandas as pd
import numpy as np


def generate_single_llm_predicted_profile(
    gene_list,
    reference_type,
    direction,
    max_retries=5
):
    """
```

```
    Generate one LLM-predicted profile:
    - Top 10 GO Biological Process-like terms
      OR
    - Top 10 KEGG-like pathways

    for either upregulated or downregulated genes.
    """

    if reference_type == "GO_Biological_Process":
        reference_instruction = """
Predict GO Biological Process-like terms.
Use biological process names such as:
cell cycle progression, regulation of Wnt signaling, cell adhesion,
immune response, ion transport, metabolic process.
Do not use KEGG-style pathway names.
"""
    elif reference_type == "KEGG_Pathway":
        reference_instruction = """
Predict KEGG pathway-like names.
Use pathway names such as:
Wnt signaling pathway, Cell cycle, Metabolic pathways,
Cytokine-cytokine receptor interaction, Tight junction.
Do not use broad GO-style descriptions if a pathway-style name is possible.
"""
    else:
        raise ValueError("reference_type must be GO_Biological_Process or KEGG_Pathway")

    prompt = f"""
You are a biomedical genomics expert.

Gene direction:
{direction}

Gene list:
{", ".join(gene_list)}

Task:
Predict the Top 10 {reference_type} results that are most likely represented by this gene list.

{reference_instruction}

Return valid JSON only.
Do not write any text before or after the JSON.
Do not use markdown fences.

JSON format:
[
  {{
    "rank": 1,
    "predicted_term_or_pathway": "specific biological process or pathway name",
    "supporting_genes": ["GENE1", "GENE2", "GENE3"],
    "confidence": "High",
    "biological_rationale": "One short sentence explaining why these genes support this result."
  }}
]

Rules:
- Return exactly 10 objects.
- Use only genes from the provided gene list.
- Do not invent genes.
- Each result must use 2 to 5 supporting genes.
- Do not provide p-values or adjusted p-values.
- Do not claim statistical significance.
- Confidence must be High, Medium, or Low.
- Do not use generic names such as Pathway1, Theme1, Process1, or Term1.
"""

    gene_set = set(gene_list)
    generic_pattern = re.compile(
        r"\b(Pathway|Theme|Process|Term)\s*_?\s*\d+\b",
        re.IGNORECASE
    )

    final_records = None
    start_time = time.time()

    for attempt in range(1, max_retries + 1):
```

```
        raw = get_completion(prompt, temperature=0, max_tokens=8000)
        parsed = parse_json_output(raw)

        if parsed is not None and isinstance(parsed, list) and len(parsed) == 10:
            records = []
            valid = True

            for idx, item in enumerate(parsed, 1):
                term = (
                    item.get("predicted_term_or_pathway")
                    or item.get("term")
                    or item.get("pathway")
                    or item.get("theme")
                    or ""
                )

                term = str(term).strip()

                genes = item.get("supporting_genes", [])

                if isinstance(genes, str):
                    genes = re.split(r",|;|\s+", genes)

                genes = [
                    str(g).strip()
                    for g in genes
                    if str(g).strip()
                ]

                invalid_genes = [g for g in genes if g not in gene_set]

                if (
                    term == ""
                    or generic_pattern.search(term)
                    or len(genes) < 2
                    or len(genes) > 5
                    or len(invalid_genes) > 0
                ):
                    valid = False
                    break

                confidence = str(item.get("confidence", "Low")).strip()
                if confidence not in ["High", "Medium", "Low"]:
                    confidence = "Low"

                records.append({
                    "Reference_Type": reference_type,
                    "Direction": direction,
                    "Rank": idx,
                    "LLM_Predicted_Term_or_Pathway": term,
                    "Supporting_Genes_List": genes,
                    "LLM_Supporting_Genes": ", ".join(genes),
                    "LLM_Confidence": confidence,
                    "Biological_Rationale": str(
                        item.get("biological_rationale", "")
                    ).strip()
                })

            if valid:
                final_records = records
                break


        print(
         f"LLM prediction failed for {reference_type} {direction}, "
         f"attempt {attempt}. Retrying..."
        )


        print("Parsed type:", type(parsed))
        print("Parsed length:", len(parsed) if isinstance(parsed, list) else "Not a list")
        print("Raw output preview:")
        print(str(raw)[:1500])

    runtime = time.time() - start_time

    if final_records is None:
        raise ValueError(
```

```
            f"Failed to generate valid Top 10 LLM predictions for "
            f"{reference_type} {direction}."
        )

    return pd.DataFrame(final_records), runtime


# Generate LLM-predicted Top 10 profiles separately
llm_go_up_df, llm_go_up_time = generate_single_llm_predicted_profile(
    top_up_genes,
    reference_type="GO_Biological_Process",
    direction="Upregulated"
)

llm_go_down_df, llm_go_down_time = generate_single_llm_predicted_profile(
    top_down_genes,
    reference_type="GO_Biological_Process",
    direction="Downregulated"
)

llm_kegg_up_df, llm_kegg_up_time = generate_single_llm_predicted_profile(
    top_up_genes,
    reference_type="KEGG_Pathway",
    direction="Upregulated"
)

llm_kegg_down_df, llm_kegg_down_time = generate_single_llm_predicted_profile(
    top_down_genes,
    reference_type="KEGG_Pathway",
    direction="Downregulated"
)


# Combine all LLM-predicted profiles
llm_predicted_go_kegg_profiles_df = pd.concat(
    [
        llm_go_up_df,
        llm_go_down_df,
        llm_kegg_up_df,
        llm_kegg_down_df
    ],
    ignore_index=True
)

display(llm_predicted_go_kegg_profiles_df)

print("LLM-predicted profiles generated successfully.")
print("Total LLM-predicted results:", len(llm_predicted_go_kegg_profiles_df))
print("GO Up:", len(llm_go_up_df))
print("GO Down:", len(llm_go_down_df))
print("KEGG Up:", len(llm_kegg_up_df))
print("KEGG Down:", len(llm_kegg_down_df))


# Save LLM-predicted profiles
llm_predicted_go_kegg_profiles_df.drop(
    columns=["Supporting_Genes_List"]
).to_csv(
    f"LLM_predicted_GO_KEGG_profiles_{SAFE_DATASET_NAME}.csv",
    index=False
)

print("Saved LLM-predicted GO/KEGG profiles.")
```

```
LLM prediction failed for GO_Biological_Process Upregulated, attempt 1. Retrying...
Parsed type: <class 'NoneType'>
Parsed length: Not a list
Raw output preview:
None
```

	Reference_Type	Direction	Rank	LLM_Predicted_Term_or_Pathway	Supporting_Genes_List	LLM_Supporting_Genes	LLM_C
0	GO_Biological_Process	Upregulated	1	regulation of Wnt signaling pathway	[AXIN2, RNF43, TGIF1, MYC, MET]	AXIN2, RNF43, TGIF1, MYC, MET	
1	GO_Biological_Process	Upregulated	2	cell adhesion via cadherin	[CDH3, CD44, CLDN1]	CDH3, CD44, CLDN1	
2	GO_Biological_Process	Upregulated	3	cell cycle progression	[CDK4, NPM1, MYC, ECT2]	CDK4, NPM1, MYC, ECT2	
3	GO_Biological_Process	Upregulated	4	extracellular matrix organization	[MMP7, CEMIP, CLDN1]	MMP7, CEMIP, CLDN1	
4	GO_Biological_Process	Upregulated	5	cell migration	[FOXQ1, CD44, MMP7]	FOXQ1, CD44, MMP7	
5	GO_Biological_Process	Upregulated	6	nucleotide biosynthetic process	[GART, IMPDH2, MTHFD1L]	GART, IMPDH2, MTHFD1L	

Next steps: Generate code with `llm_predicted_go_kegg_profiles_df` New interactive sheet

6	GO_Biological_Process	Upregulated	7	protein transport	[CSE1L, NUP37, RAN]	CSE1L, NUP37, RAN	
7	GO_Biological_Process	Upregulated	8	transcription factor activity	[GTF2IRD1, TBX3, TEAD4, PRMT5]	GTF2IRD1, TBX3, TEAD4, PRMT5	
8	GO_Biological_Process	Upregulated	9	glycosylation	[GALNT6, ANXA3]	GALNT6, ANXA3	
9	GO_Biological_Process	Upregulated	10	cellular response to growth factor stimulus	[MET, MYC, CD44]	MET, MYC, CD44	

```
# Phase 6.4e: Compare LLM-predicted GO/KEGG profiles with real enrichment Top 10 results

def get_real_top10_enrichment(enrichment_df):
    """
    Select real Top 10 enrichment terms/pathways by adjusted p-value.
    """
    temp = enrichment_df.copy()
    temp["Adjusted_P_value"] = pd.to_numeric(
        temp["Adjusted_P_value"],
        errors="coerce"
    )

    temp = temp.sort_values(
        by="Adjusted_P_value",
        ascending=True
    )

    return temp.head(10).copy()


# Real Top 10 enrichment results
real_go_up_top10 = get_real_top10_enrichment(go_up)
real_go_down_top10 = get_real_top10_enrichment(go_down)
real_kegg_up_top10 = get_real_top10_enrichment(kegg_up)
real_kegg_down_top10 = get_real_top10_enrichment(kegg_down)


real_enrichment_top10_map = {
    ("GO_Biological_Process", "Upregulated"): real_go_up_top10,
    ("GO_Biological_Process", "Downregulated"): real_go_down_top10,
    ("KEGG_Pathway", "Upregulated"): real_kegg_up_top10,
    ("KEGG_Pathway", "Downregulated"): real_kegg_down_top10
}


# Categorical comparison without 2/1/0 scoring
comparison_records = []
```

```
for _, row in llm_predicted_go_kegg_profiles_df.iterrows():
    reference_type = row["Reference_Type"]
    direction = row["Direction"]
    predicted_term = row["LLM_Predicted_Term_or_Pathway"]
    supporting_genes = row["Supporting_Genes_List"]

    real_df = real_enrichment_top10_map[(reference_type, direction)]

    match_result = compare_theme_to_real_go_kegg(
        theme=predicted_term,
        supporting_genes=supporting_genes,
        enrichment_df=real_df,
        adj_p_cutoff=0.10
    )

    status = match_result["GO_KEGG_Match_Status"]

    comparison_records.append({
        "Reference_Type": reference_type,
        "Direction": direction,
        "LLM_Rank": row["Rank"],
        "LLM_Predicted_Term_or_Pathway": predicted_term,
        "LLM_Supporting_Genes": row["LLM_Supporting_Genes"],
        "LLM_Confidence": row["LLM_Confidence"],
        "Match_Status": status,
        "Best_Real_Enrichment_Source": match_result.get("Best_Enrichment_Source", "None"),
        "Best_Real_Enrichment_Term": match_result.get("Best_Enrichment_Term", "None"),
        "Best_Adjusted_P_value": match_result.get("Best_Adjusted_P_value", np.nan),
        "Overlapping_Genes": ", ".join(match_result.get("Overlapping_Genes", [])),
        "Term_Keyword_Overlap": ", ".join(match_result.get("Term_Keyword_Overlap", [])),
        "Interpretation": match_result["Interpretation"]
    })


llm_predicted_vs_real_comparison_df = pd.DataFrame(comparison_records)

display(llm_predicted_vs_real_comparison_df)


# Summary by source and direction without composite scoring
llm_predicted_vs_real_summary_df = (
    llm_predicted_vs_real_comparison_df
    .groupby(["Reference_Type", "Direction"])
    .agg(
        Number_of_LLM_Predictions=("LLM_Predicted_Term_or_Pathway", "count"),
        Match_Count=("Match_Status", lambda x: (x == "Match").sum()),
        Partial_Count=("Match_Status", lambda x: (x == "Partial").sum()),
        Unsupported_Count=("Match_Status", lambda x: (x == "Unsupported").sum())
    )
    .reset_index()
)

llm_predicted_vs_real_summary_df["Partial_or_Match_Count"] = (
    llm_predicted_vs_real_summary_df["Match_Count"] +
    llm_predicted_vs_real_summary_df["Partial_Count"]
)

llm_predicted_vs_real_summary_df["Match_Rate_%"] = (
    llm_predicted_vs_real_summary_df["Match_Count"] /
    llm_predicted_vs_real_summary_df["Number_of_LLM_Predictions"] * 100
).round(2)

llm_predicted_vs_real_summary_df["Partial_or_Match_Rate_%"] = (
    llm_predicted_vs_real_summary_df["Partial_or_Match_Count"] /
    llm_predicted_vs_real_summary_df["Number_of_LLM_Predictions"] * 100
).round(2)

llm_predicted_vs_real_summary_df["Unsupported_Rate_%"] = (
    llm_predicted_vs_real_summary_df["Unsupported_Count"] /
    llm_predicted_vs_real_summary_df["Number_of_LLM_Predictions"] * 100
).round(2)

display(llm_predicted_vs_real_summary_df)


# Save categorical comparison outputs
llm_predicted_vs_real_comparison_df.to_csv(
```

```
    f"LLM_predicted_vs_real_GO_KEGG_categorical_comparison_{SAFE_DATASET_NAME}.csv",
    index=False
)

llm_predicted_vs_real_summary_df.to_csv(
    f"LLM_predicted_vs_real_GO_KEGG_categorical_summary_{SAFE_DATASET_NAME}.csv",
    index=False
)

print("Saved categorical LLM-predicted vs real GO/KEGG comparison files.")
```

	Reference_Type	Direction	LLM_Rank	LLM_Predicted_Term_or_Pathway	LLM_Supporting_Genes	LLM_Confidence	Match_Stat
0	GO_Biological_Process	Upregulated	1	regulation of Wnt signaling pathway	AXIN2, RNF43, TGIF1, MYC, MET	High	Unsupport
1	GO_Biological_Process	Upregulated	2	cell adhesion via cadherin	CDH3, CD44, CLDN1	High	Par
2	GO_Biological_Process	Upregulated	3	cell cycle progression	CDK4, NPM1, MYC, ECT2	High	Par
3	GO_Biological_Process	Upregulated	4	extracellular matrix organization	MMP7, CEMIP, CLDN1	Medium	Par
4	GO_Biological_Process	Upregulated	5	cell migration	FOXQ1, CD44, MMP7	Medium	Unsupport
5	GO_Biological_Process	Upregulated	6	nucleotide biosynthetic process	GART, IMPDH2, MTHFD1L	Medium	Ma
6	GO_Biological_Process	Upregulated	7	protein transport	CSE1L, NUP37, RAN	Medium	Ma
7	GO_Biological_Process	Upregulated	8	transcription factor activity	GTF2IRD1, TBX3, TEAD4, PRMT5	Medium	Unsupport
8	GO_Biological_Process	Upregulated	9	glycosylation	GALNT6, ANXA3	Low	Unsupport
9	GO_Biological_Process	Upregulated	10	cellular response to growth factor stimulus	MET, MYC, CD44	Medium	Unsupport
10	GO_Biological_Process	Downregulated	1	ion transport	TRPM6, SLC4A4, CNNM2, AQP8	High	Par
11	GO_Biological_Process	Downregulated	2	extracellular matrix organization	EDIL3, HAPLN1, DPT, CLDN23	High	Unsupport
12	GO_Biological_Process	Downregulated	3	regulation of cytokine-mediated signaling pathway	IL6R, NR3C1, PHLPP2	Medium	Par
13	GO_Biological_Process	Downregulated	4	regulation of smooth muscle contraction	MYLK, LPAR1, EDN3	Medium	Ma
14	GO_Biological_Process	Downregulated	5	response to oxidative stress	PRDX6, GPX3	Medium	Par
15	GO_Biological_Process	Downregulated	6	regulation of hormone secretion	PYY, GUCA2A, GUCA2B	Medium	Par
16	GO_Biological_Process	Downregulated	7	regulation of magnesium ion transport	TRPM6, CNNM2	Medium	Par
17	GO_Biological_Process	Downregulated	8	regulation of water transport	AQP8, CLDN23	Medium	Par

18	GO_Biological_Process	Downregulated	9	regulation of lipid transport	ABCG2, ABCA8, PLPP1	Medium	Par
19	GO_Biological_Process	Downregulated	10	regulation of glucose metabolism	PCK1, UGP2	Medium	Par
20	KEGG_Pathway	Upregulated	1	Wnt signaling pathway	AXIN2, RNF43, MYC, CD44, CLDN1	High	Par

Next steps: Generate code with `llm_predicted_vs_real_comparison_df` | New interactive sheet | Generate code with `llm_predicted_vs_real_summary`

21	KEGG_Pathway	Upregulated	2	Cell cycle	CDK4, MYC, NPM1, ECT2, PRMT5	High	Par
22	KEGG_Pathway	Upregulated	3	Tight junction	CLDN1, CDH3, CD44	High	Par
23	KEGG_Pathway	Upregulated	4	ECM-receptor interaction	CD44, MMP7, CLDN1, CEMIP	High	Par
24	KEGG_Pathway	Upregulated	5	Metabolic pathways	SORD, GART, IMPDH2, DARS, MTHFD1L	High	Ma
25	KEGG_Pathway	Upregulated	6	PI3K-Akt signaling pathway	MET, CD44, MYC, TRIM16, CDK4	High	Unsupport
26	KEGG_Pathway	Upregulated	7	p53 signaling pathway	CDK4, MYC, NPM1, PRMT5, TRIM16	High	Unsupport
27	KEGG_Pathway	Upregulated	8	Ribosome biogenesis in eukaryotes	NPM1, NUP37, RAN, MRPS23, PRMT5	High	Par
28	KEGG_Pathway	Upregulated	9	Proteoglycans in cancer	CD44, MMP7, CLDN1, CEMIP, CDH3	High	Par
29	KEGG_Pathway	Upregulated	10	Cell adhesion molecules	CD44, CLDN1, CDH3	High	Par
30	KEGG_Pathway	Downregulated	1	Glycerophospholipid metabolism	PLPP1, MGAT4A, UGP2	High	Par
31	KEGG_Pathway	Downregulated	2	Calcium signaling pathway	TRPM6, MYLK, LPAR1, ADCY9	High	Par
32	KEGG_Pathway	Downregulated	3	Cytokine-cytokine receptor interaction	IL6R, SECTM1	High	Par
33	KEGG_Pathway	Downregulated	4	Adipocytokine signaling pathway	IL6R, PYY, LPAR1	High	Par
34	KEGG_Pathway	Downregulated	5	ECM-receptor interaction	HAPLN1, EDIL3, DPT	High	Par
35	KEGG_Pathway	Downregulated	6	Tight junction	CLDN23, TSPAN7	High	Par

```
# Phase 6.4d: Automatic Verification of selected Best-of-K candidate

def format_hallucinated_genes(value):
    if value is None:
        return "None"
    if isinstance(value, list):
        return ", ".join(value) if value else "None"
    if str(value) == "None":
        return "None"
    return str(value)


verification_records = []

for _, row in comparison_df.iterrows():
    theme = row["LLM_Theme"]
    match_status = row["GO_KEGG_Match_Status"]
    supporting_genes = row["Supporting_Genes"]

    # Get hallucination info for the same theme
    hallucination_row = hallucination_df[
        hallucination_df["LLM_Theme"] == theme
    ]

    if not hallucination_row.empty:
        hallucination_rate = hallucination_row.iloc[0]["Hallucination_Rate_%"]
        hallucinated_genes = format_hallucinated_genes(
            hallucination_row.iloc[0]["Hallucinated_Genes"]
        )
    else:
        hallucination_rate = None
        hallucinated_genes = "Not checked"

    # Automatic verification logic
    if hallucination_rate is not None and hallucination_rate > 0:
        verification_status = "Unsupported"
        problem_found = f"Non-input genes detected: {hallucinated_genes}"
        corrected_comment = "This theme should be revised because at least one supporting gene was not present in the input gen

    elif match_status == "Match":
        verification_status = "Supported"
        problem_found = "None"
        corrected_comment = "All supporting genes were present in the input lists, and the theme showed direct GO/KEGG-related

    elif match_status == "Partial":
        verification_status = "Partially Supported"
        problem_found = "Only partial or indirect GO/KEGG support was detected."
        corrected_comment = "All supporting genes were present in the input lists, but the biological theme should be interpret

    else:
        verification_status = "Unsupported"
        problem_found = "No clear GO/KEGG keyword-level support was detected."
        corrected_comment = "The theme should not be treated as a final supported interpretation without additional evidence."

    verification_records.append({
        "LLM_Theme": theme,
        "Supporting_Genes": supporting_genes,
        "Gene_Presence_Check": "Passed" if hallucinated_genes == "None" else "Failed",
        "GO_KEGG_Match_Status": match_status,
        "Verification_Status": verification_status,
        "Problem_Found": problem_found,
        "Corrected_Comment": corrected_comment
    })


verification_df = pd.DataFrame(verification_records)

print("Automatic verification results:")
display(verification_df)
```

```
Automatic verification results:
```

	LLM_Theme	Supporting_Genes	Gene_Presence_Check	GO_KEGG_Match_Status	Verification_Status	Problem_Found	Correct
0	Wnt signaling activation	AXIN2, RNF43, MYC, TBX3	Passed	Partial	Partially Supported	Only partial or indirect GO/KEGG support was d...	All supp were p
1	Epithelial-mesenchymal transition and invasion	CDH3, CD44, MMP7, S100A11	Passed	Partial	Partially Supported	Only partial or indirect GO/KEGG support was d...	All supp were p
2	Cell cycle progression and proliferation	CDK4, NPM1, MYC, ECT2	Passed	Partial	Partially Supported	Only partial or indirect GO/KEGG support was d...	All supp were p
3	Stem-cell-like properties and self-renewal	CD44, AXIN2, MYC, FOXQ1	Passed	Partial	Partially Supported	Only partial or indirect GO/KEGG support was d...	All supp were p
4	Inflammatory cytokine signaling downregulation	IL6R, PHLPP2, ADAMDEC1, LPAR1	Passed	Partial	Partially Supported	Only partial or indirect GO/KEGG support was d...	All supp were p
5	Metabolic reprogramming and nucleotide synthesis	GART, IMPDH2, MTHFD1L, TRAP1	Passed	Match	Supported	None	All supp were p

Next steps: Generate code with `verification_df` New interactive sheet

```
# Phase 6.5: Theme Stability Score

import pandas as pd
import re

def normalize_theme(theme):
    """
    Convert different theme names into common biological categories.
    """
    theme_lower = theme.lower()

    # 1. Wnt / beta-catenin signaling
    if (
        "wnt" in theme_lower or
        "β-catenin" in theme_lower or
        "beta-catenin" in theme_lower or
        "catenin" in theme_lower
    ):
        return "Wnt/β-catenin signaling"

    # 2. EMT / adhesion / remodeling
    if (
        "adhesion" in theme_lower or
        "emt" in theme_lower or
        "epithelial-mesenchymal" in theme_lower or
        "junction" in theme_lower or
        "ecm" in theme_lower or
        "extracellular matrix" in theme_lower or
        "remodeling" in theme_lower or
        "invasion" in theme_lower or
        "migration" in theme_lower or
        "polarity" in theme_lower or
        "matrix metalloproteinase" in theme_lower
    ):
        return "EMT / adhesion / remodeling"

    # 3. Cell cycle / proliferation
    if (
        "cell cycle" in theme_lower or
        "cell-cycle" in theme_lower or
        "proliferation" in theme_lower or
        "mitotic" in theme_lower or
        "mitosis" in theme_lower or
        "division" in theme_lower or
```

```
            "g2/m" in theme_lower
        ):
            return "Cell-cycle / proliferation"

        # 4. Metabolic reprogramming
        if (
            "metabolic" in theme_lower or
            "metabolism" in theme_lower or
            "glycolysis" in theme_lower or
            "gluconeogenesis" in theme_lower or
            "fructose" in theme_lower or
            "mannose" in theme_lower or
            "amino acid" in theme_lower or
            "biosynthesis" in theme_lower or
            "biosynthetic" in theme_lower or
            "serine" in theme_lower or
            "glycine" in theme_lower or
            "threonine" in theme_lower or
            "redox" in theme_lower or
            "oxidative" in theme_lower or
            "pentose" in theme_lower or
            "glucuronate" in theme_lower or
            "vitamin" in theme_lower
        ):
            return "Metabolic reprogramming"

        # 5. Epithelial barrier / transport
        if (
            "barrier" in theme_lower or
            "transport" in theme_lower or
            "epithelial differentiation" in theme_lower or
            "intestinal barrier" in theme_lower or
            "chloride" in theme_lower or
            "sodium" in theme_lower or
            "tight junction" in theme_lower or
            re.search(r"\bion\b", theme_lower)
        ):
            return "Epithelial barrier / transport"

        # 6. Immune / inflammatory signaling
        if (
            "immune" in theme_lower or
            "inflammatory" in theme_lower or
            "inflammation" in theme_lower or
            "cytokine" in theme_lower or
            "interferon" in theme_lower or
            "leukocyte" in theme_lower
        ):
            return "Immune / inflammatory signaling"

        return "Other"


    # Extract themes from API Best-of-K candidates
    theme_records = []

    for candidate in api_candidates:
        candidate_id = candidate["candidate_id"]
        parsed = candidate["parsed_output"]

        if parsed is not None:
            for item in parsed:
                raw_theme = item.get("theme", "")
                normalized_theme = normalize_theme(raw_theme)

                theme_records.append({
                    "Candidate": candidate_id,
                    "Raw_Theme": raw_theme,
                    "Normalized_Theme": normalized_theme
                })

    theme_records_df = pd.DataFrame(theme_records)

    print("Raw themes mapped to normalized categories:")
    display(theme_records_df)

    # Count in how many candidates each normalized theme appeared
```

```
stability_results = []

for theme in theme_records_df["Normalized_Theme"].unique():
    candidate_count = theme_records_df[
        theme_records_df["Normalized_Theme"] == theme
    ]["Candidate"].nunique()

    stability_score = candidate_count / K * 100

    if stability_score == 100:
        stability_label = "High"
    elif stability_score >= 66:
        stability_label = "Moderate"
    elif stability_score >= 33:
        stability_label = "Low"
    else:
        stability_label = "Very Low"

    stability_results.append({
        "Normalized_Theme": theme,
        "Candidate_Frequency": f"{candidate_count}/{K}",
        "Stability_Score_%": round(stability_score, 2),
        "Stability_Label": stability_label
    })

theme_stability_df = pd.DataFrame(stability_results).sort_values(
    by="Stability_Score_%",
    ascending=False
)

print("\nTheme Stability Summary:")
theme_stability_df
```

```
Raw themes mapped to normalized categories:
```

	Candidate	Raw_Theme	Normalized_Theme
0	Candidate_1	Wnt/β-catenin pathway activation	Wnt/β-catenin signaling
1	Candidate_1	Epithelial-mesenchymal transition and cell adh...	EMT / adhesion / remodeling
2	Candidate_1	Matrix metalloproteinase-mediated extracellula...	EMT / adhesion / remodeling
3	Candidate_1	Cell cycle progression and proliferation	Cell-cycle / proliferation
4	Candidate_1	Nucleotide biosynthesis and metabolic reprogra...	Metabolic reprogramming
5	Candidate_1	Dysregulated ion transport (up and down)	Epithelial barrier / transport
6	Candidate_1	Suppression of inflammatory cytokine signaling	Immune / inflammatory signaling
7	Candidate_1	Cancer stem cell-like phenotype	Other
8	Candidate_1	Growth factor receptor-mediated signaling	Other
9	Candidate_1	Loss of phosphatase-mediated apoptosis inhibition	Other
10	Candidate_2	Wnt signaling activation	Wnt/β-catenin signaling
11	Candidate_2	Epithelial-mesenchymal transition and invasion	EMT / adhesion / remodeling
12	Candidate_2	Cell cycle progression and proliferation	Cell-cycle / proliferation
13	Candidate_2	Stem-cell-like properties and self-renewal	Other
14	Candidate_2	Inflammatory cytokine signaling downregulation	Immune / inflammatory signaling
15	Candidate_2	Metabolic reprogramming and nucleotide synthesis	Metabolic reprogramming
16	Candidate_2	Cell adhesion and tight junction integrity	EMT / adhesion / remodeling
17	Candidate_2	Extracellular matrix remodeling and invasion	EMT / adhesion / remodeling
18	Candidate_2	Ion transport dysregulation	Epithelial barrier / transport
19	Candidate_2	Transcription factor dysregulation in adenoma	Other
20	Candidate_3	Wnt/β-catenin signaling activation	Wnt/β-catenin signaling
21	Candidate_3	Cell adhesion and EMT	EMT / adhesion / remodeling
22	Candidate_3	Matrix remodeling and invasion	EMT / adhesion / remodeling
23	Candidate_3	Cell cycle progression	Cell-cycle / proliferation
24	Candidate_3	Growth factor receptor signaling	Other
25	Candidate_3	Metabolic reprogramming	Metabolic reprogramming
26	Candidate_3	Transporter and drug resistance	Epithelial barrier / transport
27	Candidate_3	Inflammatory signaling suppression	Immune / inflammatory signaling
28	Candidate_3	Ion transport and electrolyte balance	Epithelial barrier / transport
29	Candidate_3	Cell differentiation and mucin production	Other

```
Theme Stability Summary:
```

	Normalized_Theme	Candidate_Frequency	Stability_Score_%	Stability_Label
0	Wnt/β-catenin signaling	3/3	100.0	High
1	EMT / adhesion / remodeling	3/3	100.0	High
2	Cell-cycle / proliferation	3/3	100.0	High
3	Metabolic reprogramming	3/3	100.0	High
4	Epithelial barrier / transport	3/3	100.0	High
5	Immune / inflammatory signaling	3/3	100.0	High
6	Other	3/3	100.0	High

Next steps: Generate code with `theme_records_df` | New interactive sheet | Generate code with `theme_stability_df` | New interactive sheet

```
# Phase 6.6: GO/KEGG Significance Caution Flag

import pandas as pd

def add_significance_flag(df, source_name):
```

```
    """
    Add significance caution labels based on Adjusted P-value.
    """
    df = df.copy()
    df["Source"] = source_name

    def classify_significance(adj_p):
        if adj_p < 0.05:
            return "Significant after correction"
        elif adj_p < 0.10:
            return "Suggestive / borderline"
        else:
            return "Not significant after correction"

    df["Significance_Caution"] = df["Adjusted P-value"].apply(classify_significance)

    return df


# Load GO/KEGG enrichment result files
go_up = pd.read_csv(f"/content/GO-up-{DATASET_NAME}.txt", sep="\t")
go_down = pd.read_csv(f"/content/GO-down-{DATASET_NAME}.txt", sep="\t")

kegg_up = pd.read_csv(f"/content/KEGG-up-{DATASET_NAME}.txt", sep="\t")
kegg_down = pd.read_csv(f"/content/KEGG-down-{DATASET_NAME}.txt", sep="\t")



# Add significance caution flags
go_up_flagged = add_significance_flag(go_up, "GO Upregulated")
go_down_flagged = add_significance_flag(go_down, "GO Downregulated")
kegg_up_flagged = add_significance_flag(kegg_up, "KEGG Upregulated")
kegg_down_flagged = add_significance_flag(kegg_down, "KEGG Downregulated")


# Combine all enrichment results
all_enrichment_flagged = pd.concat(
    [go_up_flagged, go_down_flagged, kegg_up_flagged, kegg_down_flagged],
    ignore_index=True
)


# Count significance categories by source
significance_summary = (
    all_enrichment_flagged
    .groupby(["Source", "Significance_Caution"])
    .size()
    .reset_index(name="Number_of_Terms")
)


print("Significance caution summary:")
display(significance_summary)


# Display top terms from each source with caution labels
top_flagged_terms = (
    all_enrichment_flagged
    .sort_values(["Source", "Adjusted P-value"])
    .groupby("Source")
    .head(10)
    [["Source", "Term", "Adjusted P-value", "Significance_Caution", "Genes"]]
)

print("\nTop enrichment terms with significance caution flags:")
top_flagged_terms
```

Significance caution summary:

	Source	Significance_Caution	Number_of_Terms
0	GO Downregulated	Not significant after correction	495
1	GO Downregulated	Significant after correction	1
2	GO Downregulated	Suggestive / borderline	1
3	GO Upregulated	Not significant after correction	111
4	GO Upregulated	Significant after correction	10
5	GO Upregulated	Suggestive / borderline	128
6	KEGG Downregulated	Not significant after correction	114
7	KEGG Downregulated	Significant after correction	1

Next steps: Generate code with significance_summary | New interactive sheet | Generate code with top_flagged_terms | New interactive sheet

```
# Separate GO and KEGG enrichment results
# Top 10 Upregulated + Top 10 Downregulated for each source

TOP_N = 10

display_cols = ["Source", "Term", "Adjusted P-value", "Significance_Caution", "Genes"]

# GO Biological Process terms
go_up_top10 = (
    go_up_flagged
    .sort_values("Adjusted P-value", ascending=True)
    .head(TOP_N)
    [display_cols]
)

go_down_top10 = (
    go_down_flagged
    .sort_values("Adjusted P-value", ascending=True)
    .head(TOP_N)
    [display_cols]
)

go_top20_terms = pd.concat(
    [go_up_top10, go_down_top10],
    ignore_index=True
)

# KEGG pathway terms
kegg_up_top10 = (
    kegg_up_flagged
    .sort_values("Adjusted P-value", ascending=True)
    .head(TOP_N)
    [display_cols]
)

kegg_down_top10 = (
    kegg_down_flagged
    .sort_values("Adjusted P-value", ascending=True)
    .head(TOP_N)
    [display_cols]
)

kegg_top20_pathways = pd.concat(
    [kegg_up_top10, kegg_down_top10],
    ignore_index=True
)

print("GO Biological Process terms: Top 10 Upregulated + Top 10 Downregulated")
print("Total GO terms displayed:", len(go_top20_terms))
display(go_top20_terms)

print("\nKEGG pathways: Top 10 Upregulated + Top 10 Downregulated")
print("Total KEGG pathways displayed:", len(kegg_top20_pathways))
display(kegg_top20_pathways)
```

7	GO Upregulated	Regulation of Protein Import Into Nucleus (GO:...	0.037639	Significant after correction	ECT2;RAN
8	GO Upregulated	Positive Regulation of Nucleocytoplasmic Trans...	0.039043	Significant after correction	ECT2;RAN
9	GO Upregulated	Purine Ribonucleotide Biosynthetic Process (GO...	0.039043	Significant after correction	IMPDH2;GART

```
GO Biological Process terms: Top 10 Upregulated + Top 10 Downregulated
Total GO terms displayed: 20
```

	Source	Term	Adjusted P-value	Significance_Caution	Genes
771	KEGG Downregulated	BILE SECRETION	0.006812	Significant after correction	ADCY9;AQP8;SLC4A4;ABCG2
772	KEGG Downregulated	PROXIMAL TUBULE BICARBONATE RECLAMATION	0.066726	Suggestive / borderline	PCK1;SLC4A4
0	GO Upregulated	GTP Metabolic Process (GO:0046039)	0.029288	Significant after correction	IMPDH2;RAN
773	GO Downregulated	CONTRACTION	0.122567	Not significant after correction	ADCY9;EDN3;MYLK
1	GO Upregulated	Protein Export From Nucleus (GO:0006611)	0.029288	Significant after correction	CSE1L;RAN
2	GO Upregulated	MiRNA-mediated Post-Transcriptional Gene Silen...	0.029288	Significant after correction	AJUBA;RAN
774	KEGG Downregulated	PHOSPHOLIPASE D SIGNALING PATHWAY	0.122567	Not significant after correction	ADCY9;LPAR1;PLPP1
775	GO Upregulated	Bicellular Tight Junction Assembly (GO:0070830)	0.029288	Significant after correction	ECT2;CLDN1
3	GO Upregulated	Positive Regulation of Protein Import Into Nuc...	0.029288	Significant after correction	ECT2;RAN
4	GO Upregulated	Apical Junction Assembly (GO:0043297)	0.035402	Significant after correction	ECT2;CLDN1
5	GO Upregulated	Tight Junction Assembly (GO:0120192)	0.035402	Significant after correction	ECT2;CLDN1
6	GO Upregulated	Regulation of Protein Import Into Nucleus (GO:...	0.037639	Significant after correction	ECT2;RAN
7	GO Upregulated	Positive Regulation of Nucleocytoplasmic Trans...	0.039043	Significant after correction	ECT2;RAN
8	GO Upregulated	Purine Ribonucleotide Biosynthetic Process (GO...	0.039043	Significant after correction	IMPDH2;GART
9	GO Upregulated	Tonic Smooth Muscle Contraction (GO:0014820)	0.027902	Significant after correction	EDN3;MYLK
10	GO Downregulated	Steroid Hormone Receptor Signaling Pathway (GO...	0.076029	Suggestive / borderline	NR3C1;PLPP1
11	GO Downregulated	Regulation of Leukocyte Chemotaxis (GO:0002688)	0.104623	Not significant after correction	EDN3;IL6R
12	GO Downregulated	Xenobiotic Transport (GO:0042908)	0.104623	Not significant after correction	ABCA8;ABCG2
13	GO Downregulated	Metal Ion Transport (GO:0030001)	0.131571	Not significant after correction	SLC4A4;TRPM6;SLC17A4
14	GO Downregulated	Adiponectin-activated Signaling Pathway (GO:00...	0.131571	Not significant after correction	APPL2
15	GO Downregulated	Regulation of SA Node Cell Action Potential (G...	0.131571	Not significant after correction	ANK2
16	GO Downregulated	Glyoxylate Metabolic Process (GO:0046487)	0.131571	Not significant after correction	MGAT4A
17	GO Downregulated	Glycoprotein Metabolic Process (GO:0009100)	0.131571	Not significant after correction	MGAT4A;GCNT2
18	GO Downregulated	Sphingolipid Metabolic Process (GO:0006665)	0.131571	Not significant after correction	PLPP1;ABCG2
19	GO Downregulated	Sphingolipid Metabolic Process (GO:0006665)	0.131571	Not significant after correction	PLPP1;ABCG2

```
KEGG pathways: Top 10 Upregulated + Top 10 Downregulated
Total KEGG pathways displayed: 20
```

	Source	Term	Adjusted P-value	Significance_Caution	Genes
0	KEGG Upregulated	NUCLEOCYTOPLASMIC TRANSPORT	0.006752	Significant after correction	CSE1L;RAN;NUP37
1	KEGG Upregulated	HIPPO SIGNALING PATHWAY - MULTIPLE SPECIES	0.006866	Significant after correction	TEAD4;AJUBA
2	KEGG Upregulated	ONE CARBON POOL BY FOLATE	0.007067	Significant after correction	MTHFD1L;GART
3	KEGG Upregulated	RIBOSOME BIOGENESIS IN EUKARYOTES	0.023286	Significant after correction	NHP2;RAN
4	KEGG Upregulated	PURINE METABOLISM	0.048130	Significant after correction	IMPDH2;GART
5	KEGG Upregulated	HIPPO SIGNALING PATHWAY	0.061371	Suggestive / borderline	TEAD4;AJUBA
6	KEGG Upregulated	CORNIFIED ENVELOPE FORMATION	0.096239	Suggestive / borderline	CLDN1;S100A11

Next steps: Generate code with go_top20_terms | New interactive sheet | Generate code with kegg_top20_pathways | New interactive sheet

```
# Save separated GO and KEGG top results

go_up_top10.to_csv(
    f"GO_upregulated_top10_terms_{SAFE_DATASET_NAME}.csv",
    index=False
)

go_down_top10.to_csv(
    f"GO_downregulated_top10_terms_{SAFE_DATASET_NAME}.csv",
    index=False
)

kegg_up_top10.to_csv(
    f"KEGG_upregulated_top10_pathways_{SAFE_DATASET_NAME}.csv",
    index=False
)

kegg_down_top10.to_csv(
    f"KEGG_downregulated_top10_pathways_{SAFE_DATASET_NAME}.csv",
    index=False
)

go_top20_terms.to_csv(
    f"GO_top20_terms_up_down_{SAFE_DATASET_NAME}.csv",
    index=False
)

kegg_top20_pathways.to_csv(
    f"KEGG_top20_pathways_up_down_{SAFE_DATASET_NAME}.csv",
    index=False
)

print("Separated GO and KEGG top results were saved successfully.")
```

```
Separated GO and KEGG top results were saved successfully.
```

```
# Automatically summarize theme stability results

def summarize_theme_stability(theme_stability_df):
    high_themes = theme_stability_df[
        theme_stability_df["Stability_Label"] == "High"
    ]["Normalized_Theme"].tolist()

    moderate_themes = theme_stability_df[
        theme_stability_df["Stability_Label"] == "Moderate"
    ]["Normalized_Theme"].tolist()

    low_themes = theme_stability_df[
        theme_stability_df["Stability_Label"] == "Low"
    ]["Normalized_Theme"].tolist()

    very_low_themes = theme_stability_df[
        theme_stability_df["Stability_Label"] == "Very Low"
    ]["Normalized_Theme"].tolist()

    if high_themes:
        main_result = "High stability themes: " + ", ".join(high_themes)
    else:
        main_result = "No high-stability themes detected"

    interpretation_parts = []

    if high_themes:
        interpretation_parts.append(
            "High-stability themes appeared across all candidates: "
            + ", ".join(high_themes)
            + "."
        )

    if moderate_themes:
        interpretation_parts.append(
            "Moderate-stability themes appeared in most candidates: "
            + ", ".join(moderate_themes)
            + "."
        )
```

```
    if low_themes:
        interpretation_parts.append(
            "Low-stability themes appeared less consistently: "
            + ", ".join(low_themes)
            + "."
        )

    if very_low_themes:
        interpretation_parts.append(
            "Very-low-stability themes were rarely recovered: "
            + ", ".join(very_low_themes)
            + "."
        )

    interpretation = " ".join(interpretation_parts)

    return main_result, interpretation
```

```
verification_counts = verification_df["Verification_Status"].value_counts().to_dict()

verification_main_result = "; ".join(
    [f"{status}: {count}" for status, count in verification_counts.items()]
)
```

```
# Phase 6.7: Final Evaluation Summary

# Build GO/KEGG categorical summary text
if "go_kegg_summary_df" in globals() and len(go_kegg_summary_df) > 0:
    go_kegg_row = go_kegg_summary_df.iloc[0]

    go_kegg_main_result = (
        f"Match = {int(go_kegg_row['Match_Count'])}, "
        f"Partial = {int(go_kegg_row['Partial_Count'])}, "
        f"Unsupported = {int(go_kegg_row['Unsupported_Count'])}; "
        f"Partial-or-Match rate = {go_kegg_row['Partial_or_Match_Rate_%']}%"
    )
else:
    go_kegg_main_result = "GO/KEGG categorical support summary was not available."


# Build hallucination result text
if "overall_hallucination_rate" in globals():
    hallucination_main_result = f"{round(overall_hallucination_rate, 2)}%"
else:
    hallucination_main_result = "Not calculated"


# Build selected candidate result text
if "selected_candidate_id" in globals():
```